Cairo University
Faculty of Computers and Artificial Intelligence
Machine Learning Course

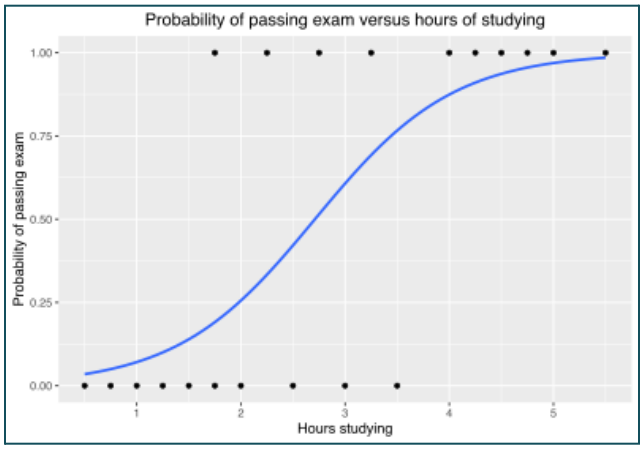
Lab 4: Logistic Regression

Lab Outline:

- Summary of logistic regression
- Pseudocode for gradient descent in logistic regression
- Steps of applying logistic regression to a dataset
  - Example
  - Data preprocessing
- Practice

Summary of Logistic Regression:

Logistic regression is a ***supervised*** machine learning algorithm that is used for ***classification*** tasks. In classification tasks, the dependent variable (target) is categorical (i.e. has discrete classes). Logistic regression builds a model that estimates the probability of a class occurring based on some independent variables.



The ***hypothesis function*** of logistic regression, which predicts the probability that an observation in the data belongs to a certain class, needs to be a function that produces values between 0 and 1. Thus, the sigmoid function can be used:

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

In ***binary logistic regression,*** the target has only two possible outcomes (e.g. 0 or 1). In this case, to perform classification, an estimated probability that is less than 0.5 will predict class 0 while an estimated probability greater than 0.5 will predict class 1.

The cost function used in binary logistic regression is the ***binary cross-entropy*** (log loss) which is calculated as follows:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\left[-y^{(i)}\log(h_\theta(x^{(i)})) - (1-y^{(i)})\log(1-h_\theta(x^{(i)}))\right]$$

Gradient descent can be used to get the parameters that minimize the cost function.

Pseudocode for Gradient Descent in Logistic Regression:

Function fit_GD

Input: *X, y*
Output: *theta*

Initialize *theta* array
Set *alpha* **and** *max_iterations*
for *i=0* **to** *max_iterations*
    Compute the hypothesis function values: *h=sigmoid(theta.T*X)*
    for *j=0* **to** *theta.length*
        Compute the partial derivative of the error: *par_der=(1/m)*sum((h-y)*X_j)*
        Update $theta_j$ according to the equation: $theta_j$=$theta_j$-*alpha*par_der*
    end
end

Steps of Applying Logistic Regression to a Dataset:

1. **Analyze** the dataset
2. Perform data **preprocessing**
3. **Split** the data into train and test sets
4. **Fit** the logistic regression model to the training data
5. **Evaluate** the fitted model on the test data
6. Generate **predictions** for new data

*Note: **Some data preprocessing techniques should be performed after the splitting of data.** So, we can swap the orders of steps 2 and 3, or perform step 3 at a certain point within step 2.*

Example:

Suppose you are the administrator of a university department and you want to determine each applicant's chance of acceptance based on their results on two exams. You have data from previous applicants which consists of the applicants' scores on two exams and the admissions decision.

You can use logistic regression to build a classification model that estimates an applicant's probability of acceptance based on the scores from those two exams.

Note: The code can be found inside "Lab 4.py".

Data Preprocessing:

Data preprocessing is the process of preparing raw data to make it suitable for a machine learning model. This process depends on the type of data (text, images, etc.) and the dataset itself since each dataset is different. Data preprocessing of CSV data may include:

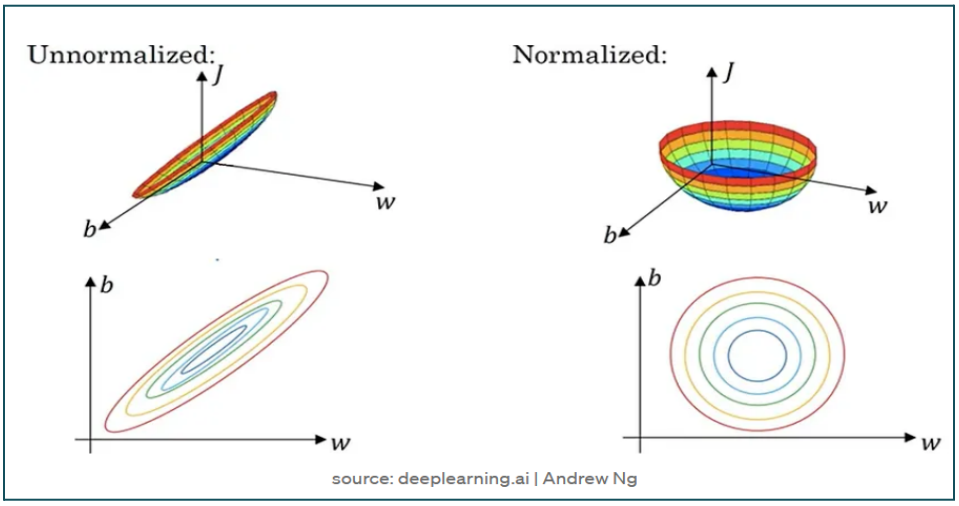
- **Handling missing data** by removing it, or replacing it with the mean or mode, ...

- **Encoding categorical data** to encode categorical features and labels with numerical values in order to be suitable for the model. Here is an example of sklearn's LabelEncoder:

```
>>> le = LabelEncoder()
>>> le.fit(["paris", "paris", "tokyo", "amsterdam"])
LabelEncoder()
>>> list(le.classes_)
['amsterdam', 'paris', 'tokyo']
>>> le.transform(["tokyo", "tokyo", "paris"])
array([2, 2, 1]...)
>>> list(le.inverse_transform([2, 2, 1]))
['tokyo', 'tokyo', 'paris']
```

- **Feature scaling** transforms the values of features in the dataset to a similar scale to reduce the impact of outliers and ensure that the data is on the same scale. This in turn ensures that features contribute equally to the model with no domination of features with larger values. Feature scaling methods which include standardization and normalization can be helpful especially with normally distributed data such as the heights of people, or even the prices of vegetables in the market.

  By looking at the next image, we can see that the unnormalized contour (plot of a three-dimensional surface on a two-dimensional plane) is skewed and assumes the shape of an oval. The image next to it depicts a normalized contour that is shaped like a circle and is uniformly spaced. When we use gradient descent in both cases, we notice that if the input is normalized, gradient descent converges to the minimum faster. If the input is not normalized, gradient descent takes many steps before converging to a minimum, which might slow down the model's learning process.



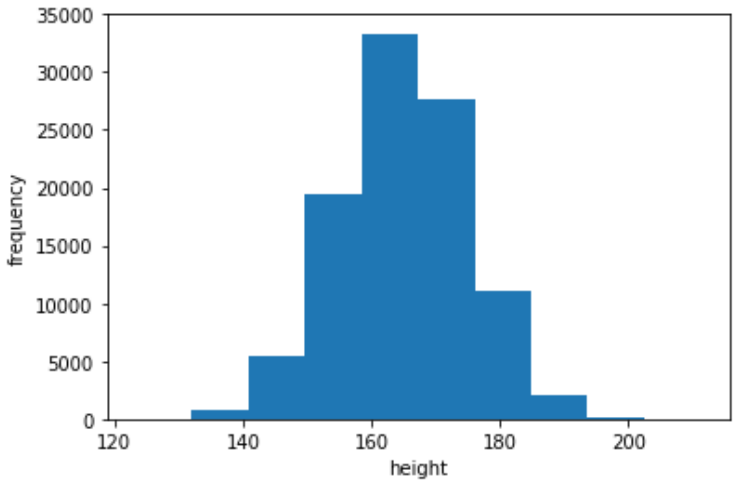
Case Study:

Suppose that we measured the height of 100,000 people, and we found that the mean of their heights is 165 cm and the standard deviation is 10 cm. So, we can generate this experiment's data and plot it as the follows:

```
import numpy as np
import matplotlib.pyplot as plt

mean, sd ,size = (165, 10, 100000)
x = np.random.normal(mean, sd, size)

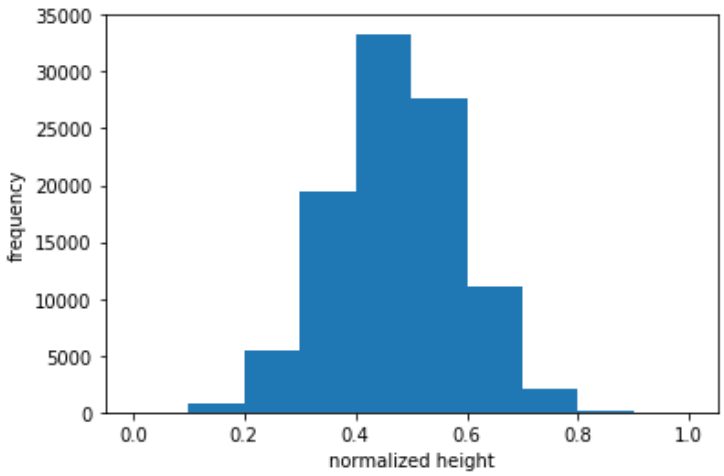
plt.hist(x)
plt.xlabel('height')
plt.ylabel('frequency')
plt.show()
```



Normalization (Min-Max Scaling) is used to transform data to be on a similar scale with range [0, 1]. The new points are calculated as follows:

$$x_{norm}^{(i)} = \frac{x^{(i)} - \mathbf{x}_{min}}{\mathbf{x}_{max} - \mathbf{x}_{min}}$$

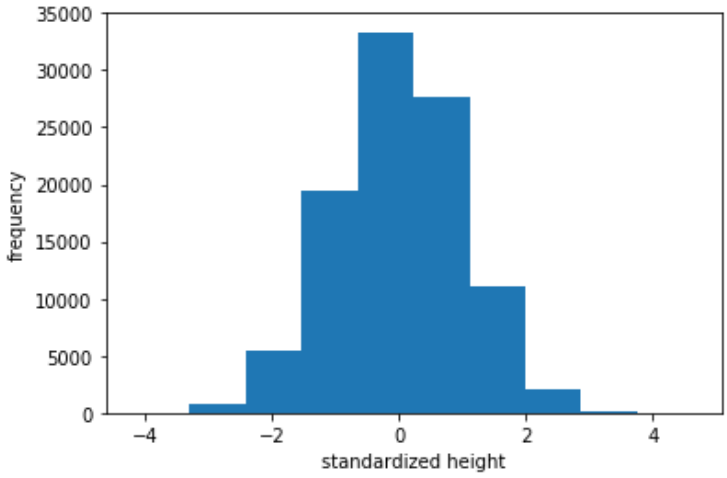
```
x_range = max(x)-min(x)
x_normalized = (x-min(x))/x_range
plt.hist(x_normalized)
plt.xlabel('normalized height')
plt.ylabel('frequency')
plt.show()
```



Standardization (Z-Score Normalization) is the transformation of features by subtracting from mean and dividing by standard deviation. The new points are calculated as follows:

$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x}$$

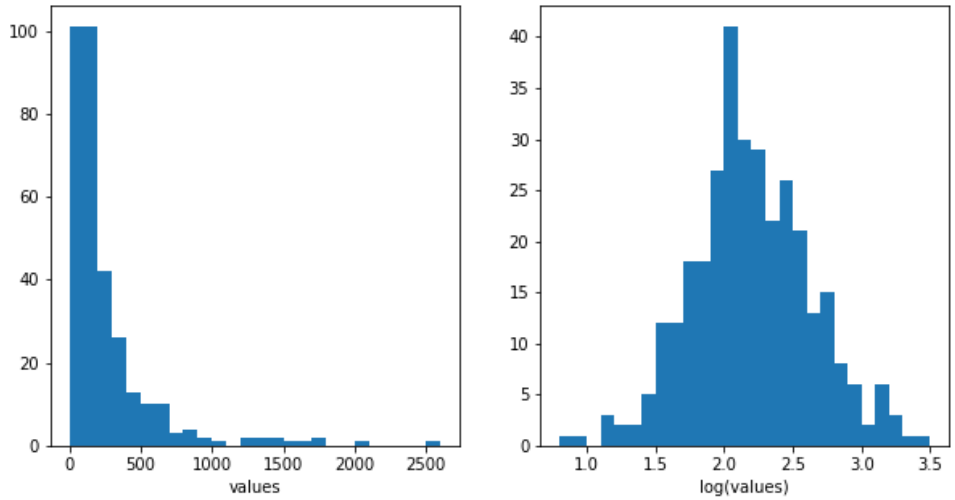
```
x_standardized = (x-np.mean(x))/np.std(x)
plt.hist(x_standardized)
plt.xlabel('standardized height')
plt.ylabel('frequency')
plt.show()
```



What if we have skewed data?

Skewed data degrades the model's ability (especially regression-based models) to describe typical cases as it has to deal with rare cases on extreme values. Skewed data also does not work well with many statistical methods.

One way to deal with skewed data is to perform ***log-scale transformation,*** i.e. transform the skewed distribution to be a bit similar to the normal distribution. Note that there is no log for zero or negative values (so, add a constant to all values to ensure values are greater than one).



Practice:

Given "acceptance_data.txt" and "Lab 4.py", solve the tasks in the code.